

```
In [ ]: !pip install nbconvert
```

```
In [1]: # This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input direc

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you c
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
```

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load training and testing datasets
train_path = "UNSW_NB15_training-set.csv"
test_path = "UNSW_NB15_testing-set.csv"

train_data = pd.read_csv(train_path)
test_data = pd.read_csv(test_path)

# Display basic info
print("Training Data Info:")
print(train_data.info())

print("\nSample Training Data:")
print(train_data.head())

# Check for missing values
print("\nMissing Values:")
print(train_data.isnull().sum())

# Plot class distribution
plt.figure(figsize=(8, 5))
sns.countplot(data=train_data, x="label", palette="viridis")
plt.title("Class Distribution in Training Data (0: Normal, 1: Malicious)")
plt.xlabel("Label")
plt.ylabel("Count")
plt.show()

# Visualize protocol types
plt.figure(figsize=(10, 5))
sns.countplot(data=train_data, x="proto", order=train_data["proto"].value_counts().index[:10], palette="Set2")
plt.title("Top 10 Protocol Types")
plt.xlabel("Protocol")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()

# Correlation heatmap
numeric_columns = train_data.select_dtypes(include=["float64", "int64"]).columns
correlation_matrix = train_data[numeric_columns].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=False, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

Training Data Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 82332 entries, 0 to 82331

Data columns (total 45 columns):

#	Column	Non-Null Count	Dtype
0	id	82332 non-null	int64
1	dur	82332 non-null	float64
2	proto	82332 non-null	object
3	service	82332 non-null	object
4	state	82332 non-null	object
5	spkts	82332 non-null	int64
6	dpkts	82332 non-null	int64

7	sbytes	82332	non-null	int64
8	dbytes	82332	non-null	int64
9	rate	82332	non-null	float64
10	sttl	82332	non-null	int64
11	dttl	82332	non-null	int64
12	sload	82332	non-null	float64
13	dload	82332	non-null	float64
14	sloss	82332	non-null	int64
15	dloss	82332	non-null	int64
16	sinpkt	82332	non-null	float64
17	dinpkt	82332	non-null	float64
18	sjit	82332	non-null	float64
19	djit	82332	non-null	float64
20	swin	82332	non-null	int64
21	stcpb	82332	non-null	int64
22	dtcpb	82332	non-null	int64
23	dwin	82332	non-null	int64
24	tcprtt	82332	non-null	float64
25	synack	82332	non-null	float64
26	ackdat	82332	non-null	float64
27	smean	82332	non-null	int64
28	dmean	82332	non-null	int64
29	trans_depth	82332	non-null	int64
30	response_body_len	82332	non-null	int64
31	ct_srv_src	82332	non-null	int64
32	ct_state_ttl	82332	non-null	int64
33	ct_dst_ltm	82332	non-null	int64
34	ct_src_dport_ltm	82332	non-null	int64
35	ct_dst_sport_ltm	82332	non-null	int64
36	ct_dst_src_ltm	82332	non-null	int64
37	is_ftp_login	82332	non-null	int64
38	ct_ftp_cmd	82332	non-null	int64
39	ct_flw_http_mthd	82332	non-null	int64
40	ct_src_ltm	82332	non-null	int64
41	ct_srv_dst	82332	non-null	int64
42	is_sm_ips_ports	82332	non-null	int64
43	attack_cat	82332	non-null	object
44	label	82332	non-null	int64

dtypes: float64(11), int64(30), object(4)

memory usage: 28.3+ MB

None

Sample Training Data:

	id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	\
0	1	0.000011	udp	-	INT	2	0	496	0	
1	2	0.000008	udp	-	INT	2	0	1762	0	
2	3	0.000005	udp	-	INT	2	0	1068	0	
3	4	0.000006	udp	-	INT	2	0	900	0	
4	5	0.000010	udp	-	INT	2	0	2126	0	

	rate	...	ct_dst_sport_ltm	ct_dst_src_ltm	is_ftp_login	\
0	90909.0902	...	1	2	0	
1	125000.0003	...	1	2	0	
2	200000.0051	...	1	3	0	
3	166666.6608	...	1	3	0	
4	100000.0025	...	1	3	0	

	ct_ftp_cmd	ct_flw_http_mthd	ct_src_ltm	ct_srv_dst	is_sm_ips_ports	\
0	0	0	1	2	0	
1	0	0	1	2	0	
2	0	0	1	3	0	
3	0	0	2	3	0	
4	0	0	2	3	0	

	attack_cat	label
0	Normal	0
1	Normal	0
2	Normal	0
3	Normal	0
4	Normal	0

[5 rows x 45 columns]

Missing Values:

id	0
dur	0
proto	0
service	0
state	0
spkts	0
dpkts	0
sbytes	0
dbytes	0

```

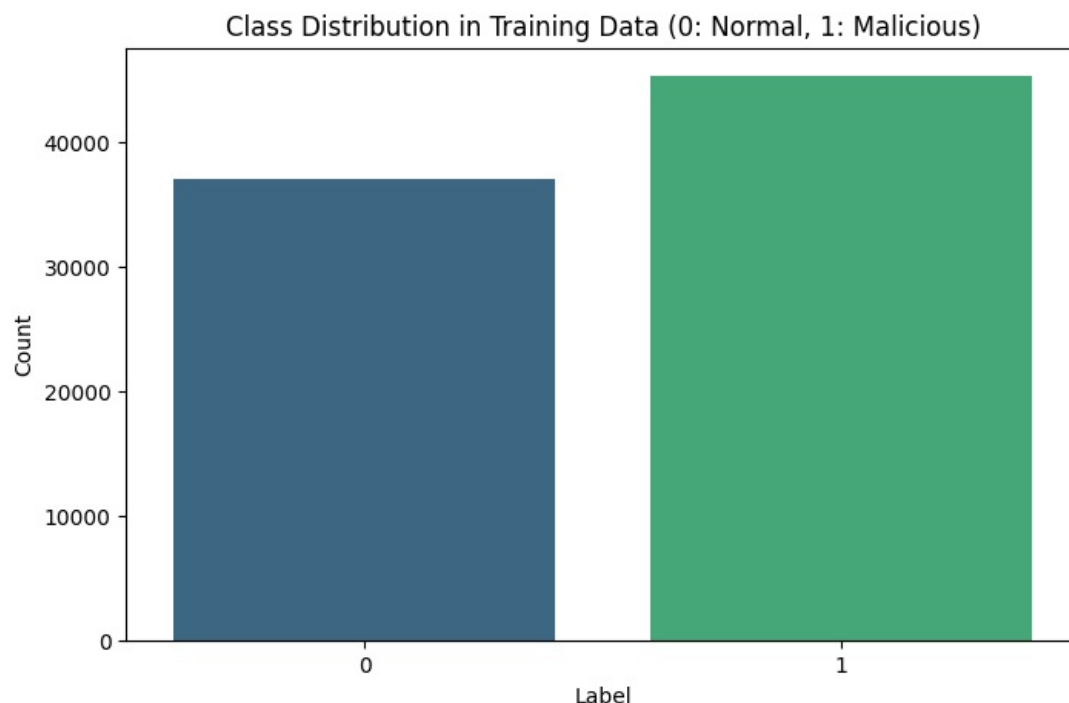
rate      0
sttl      0
dttl      0
sload     0
dload     0
sloss     0
dloss     0
sinpkt    0
dinpkt    0
sjit      0
djit      0
swin      0
stcpb     0
dtcpb     0
dwin      0
tcprtt    0
synack    0
ackdat    0
smean     0
dmean     0
trans_depth 0
response_body_len 0
ct_srv_src 0
ct_state_ttl 0
ct_dst_ltm 0
ct_src_dport_ltm 0
ct_dst_sport_ltm 0
ct_dst_src_ltm 0
is_ftp_login 0
ct_ftp_cmd 0
ct_flw_http_mthd 0
ct_src_ltm 0
ct_srv_dst 0
is_sm_ips_ports 0
attack_cat 0
label     0
dtype: int64

```

C:\Users\Bhautik\AppData\Local\Temp\ipykernel_18436\1936577200.py:25: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=train_data, x="label", palette="viridis")
```

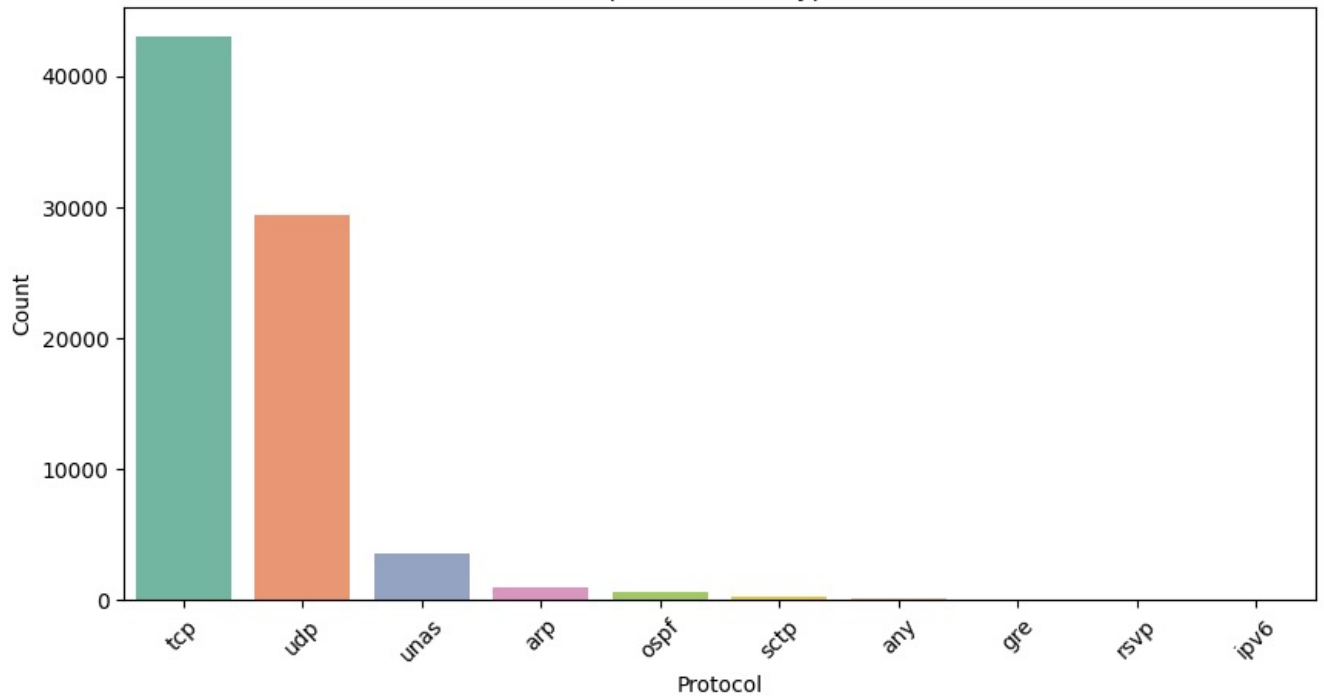


C:\Users\Bhautik\AppData\Local\Temp\ipykernel_18436\1936577200.py:33: FutureWarning:

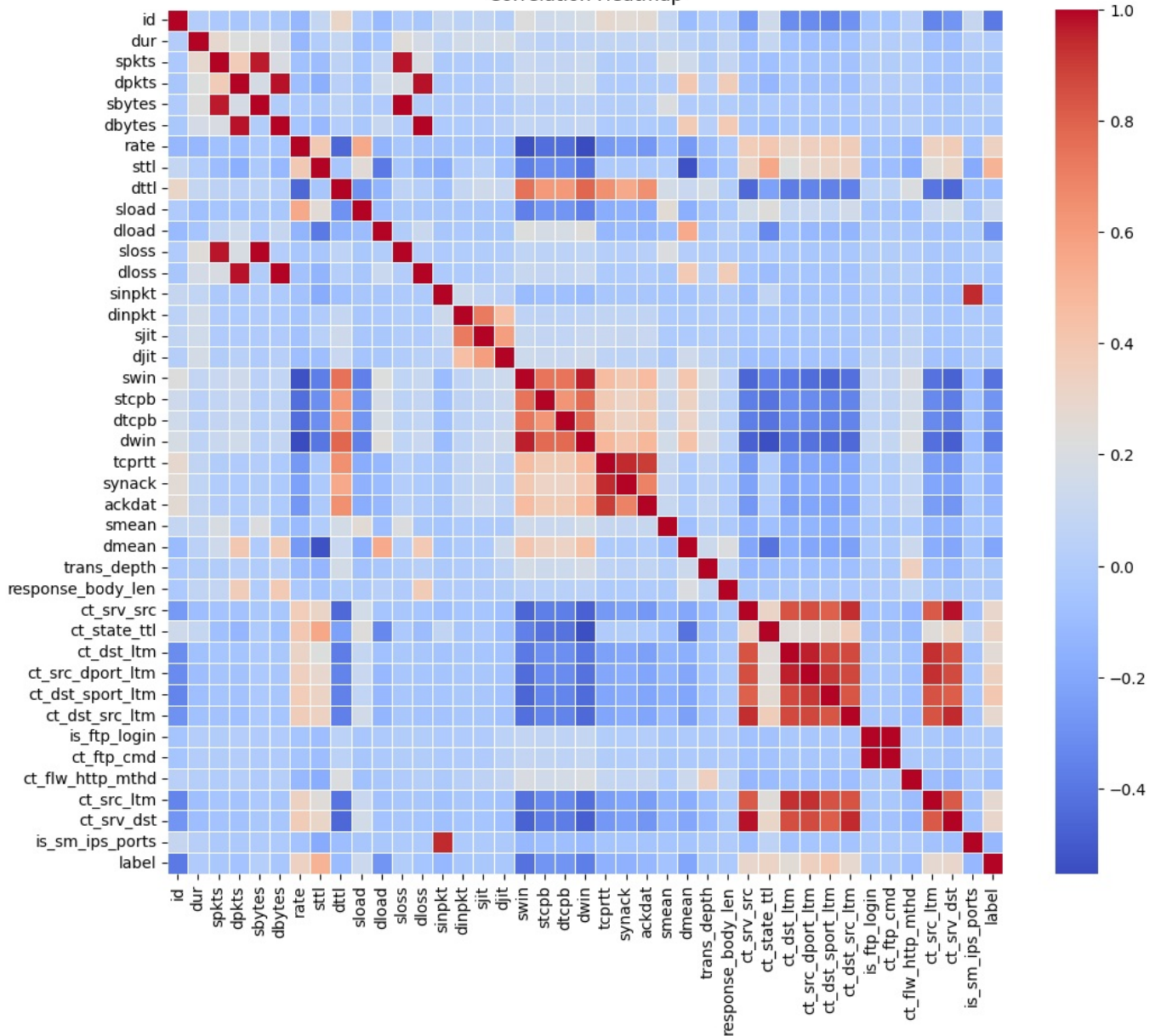
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=train_data, x="proto", order=train_data["proto"].value_counts().index[:10], palette="Set2")
```

Top 10 Protocol Types



Correlation Heatmap



In [3]: # --- Cell 4: Common Data Preparation ---

```
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
import pandas as pd

# Load training and testing datasets
train_path = "UNSW_NB15_training-set.csv"
test_path = "UNSW_NB15_testing-set.csv"

train_data = pd.read_csv(train_path)
test_data = pd.read_csv(test_path)

# Drop unnecessary columns from both datasets
train_df = train_data.drop(["id", "attack_cat"], axis=1, errors="ignore")
test_df = test_data.drop(["id", "attack_cat"], axis=1, errors="ignore")

# Identify categorical columns
categorical_cols = train_df.select_dtypes(include=['object']).columns

# Use one-hot encoding to ensure columns are aligned
combined_data = pd.concat([train_df, test_df], axis=0)
encoded_data = pd.get_dummies(combined_data, columns=categorical_cols)

# Separate the combined data back into training and testing sets
X_train_df = encoded_data.iloc[:len(train_df)]
X_test_df = encoded_data.iloc[len(train_df):]

# Separate labels
y_train = X_train_df.pop('label')
y_test = X_test_df.pop('label')

# --- Feature Scaling ---
# Fit the scaler ONLY on the training data
scaler = MinMaxScaler()
X_train = pd.DataFrame(scaler.fit_transform(X_train_df), columns=X_train_df.columns)

# Use the SAME scaler to transform the test data
X_test = pd.DataFrame(scaler.transform(X_test_df), columns=X_test_df.columns)

print("Data preprocessing complete.")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
```

Data preprocessing complete.
X_train shape: (82332, 196)
X_test shape: (175341, 196)

In [4]: # --- Cell 5: Autoencoder for Anomaly Detection ---

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Build autoencoder
autoencoder = Sequential([
    Dense(64, activation="relu", input_dim=X_train.shape[1]),
    Dense(32, activation="relu"),
    Dense(64, activation="relu"),
    Dense(X_train.shape[1], activation="sigmoid")
])

autoencoder.compile(optimizer="adam", loss="mse")
history = autoencoder.fit(X_train, X_train, epochs=10, batch_size=32, validation_split=0.2, verbose=1)

# Plot training loss
plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Autoencoder Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Reconstruction loss for anomaly detection
reconstruction_loss = np.mean(np.square(X_train.values - autoencoder.predict(X_train)), axis=1)
threshold = np.percentile(reconstruction_loss, 95) # Set threshold as 95th percentile
print(f"Anomaly Threshold: {threshold}")

# Visualize reconstruction loss
plt.figure(figsize=(8, 5))
```

```
sns.histplot(reconstruction_loss, bins=50, kde=True, color="blue")
plt.axvline(threshold, color="red", linestyle="--", label="Threshold")
plt.title("Reconstruction Loss Distribution")
plt.xlabel("Loss")
plt.ylabel("Count")
plt.legend()
plt.show()

# Mark anomalies
anomalies = (reconstruction_loss > threshold).astype(int)
print(f"Number of anomalies detected: {np.sum(anomalies)}")
```

C:\Users\Bhautik\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-pack
ages\Python311\site-packages\keras\src\export\tf2onnx_lib.py:8: FutureWarning: In the future `np.object` will be
defined as the corresponding NumPy scalar.

```
if not hasattr(np, "object"):
```

C:\Users\Bhautik\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-pack
ages\Python311\site-packages\keras\src\layers\core\dense.py:106: UserWarning: Do not pass an `input\_shape`/`input
_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first lay
er in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Epoch 1/10

2059/2059 ————— **25s** 10ms/step - loss: 0.0056 - val_loss: 0.0017

Epoch 2/10

2059/2059 ————— **21s** 10ms/step - loss: 6.4384e-04 - val_loss: 9.6716e-04

Epoch 3/10

2059/2059 ————— **20s** 10ms/step - loss: 5.4900e-04 - val_loss: 8.3982e-04

Epoch 4/10

2059/2059 ————— **15s** 7ms/step - loss: 4.9301e-04 - val_loss: 4.2114e-04

Epoch 5/10

2059/2059 ————— **14s** 7ms/step - loss: 4.5327e-04 - val_loss: 3.4556e-04

Epoch 6/10

2059/2059 ————— **12s** 6ms/step - loss: 4.3154e-04 - val_loss: 3.1371e-04

Epoch 7/10

2059/2059 ————— **21s** 10ms/step - loss: 3.9914e-04 - val_loss: 3.7447e-04

Epoch 8/10

2059/2059 ————— **20s** 10ms/step - loss: 3.6492e-04 - val_loss: 3.6437e-04

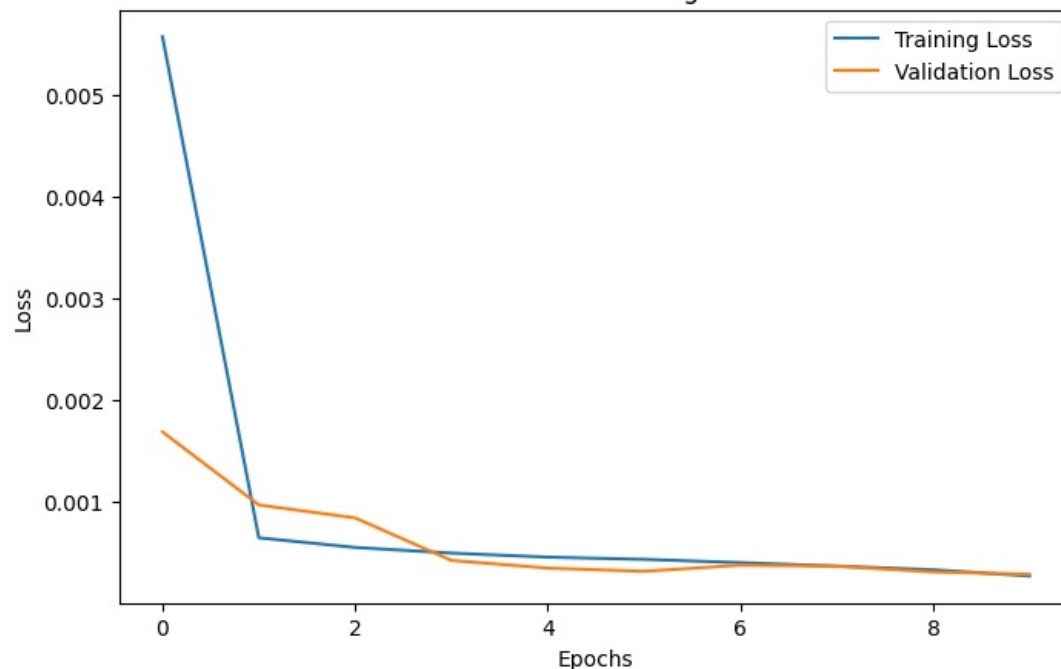
Epoch 9/10

2059/2059 ————— **18s** 9ms/step - loss: 3.2900e-04 - val_loss: 3.0757e-04

Epoch 10/10

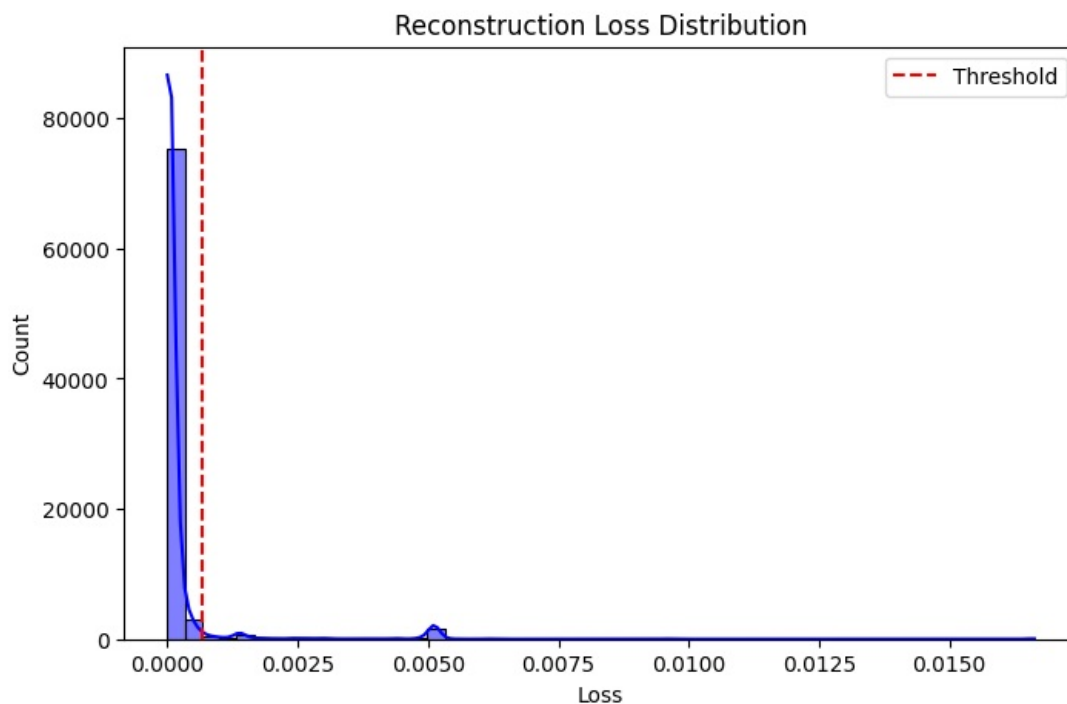
2059/2059 ————— **20s** 8ms/step - loss: 2.6805e-04 - val_loss: 2.8447e-04

Autoencoder Training Loss



2573/2573 ————— **15s** 6ms/step

Anomaly Threshold: 0.0006742298399929782



Number of anomalies detected: 4117

```
In [7]: # --- Cell 6: Random Forest Classifier and SHAP ---

!pip install shap
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix, ConfusionMatrixDisplay
import shap
import matplotlib.pyplot as plt
import numpy as np

# Train Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Make predictions
y_pred = rf_model.predict(X_test)

# Classification metrics
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Visualize confusion matrix
plt.figure(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Normal", "Malicious"])
disp.plot(cmap="Blues", values_format='d')
plt.title("Confusion Matrix - Random Forest")
plt.show()

print(f"\nAccuracy: {accuracy_score(y_test, y_pred):.4f}")

# Visualize feature importance
feature_importance = rf_model.feature_importances_
features = X_train.columns
sorted_idx = np.argsort(feature_importance)[-10:] # Top 10 features

plt.figure(figsize=(10, 6))
plt.barh(features[sorted_idx], feature_importance[sorted_idx], color="skyblue")
```

```

plt.xlabel("Feature Importance")
plt.title("Top 10 Important Features")
plt.show()

# --- SHAP Analysis ---
# Using a smaller sample for faster computation
X_test_sample = X_test.sample(n=1000, random_state=42)

explainer = shap.TreeExplainer(rf_model)

# FIX: Use explainer(X) instead of explainer.shap_values(X)
# This returns a shap.Explanation object which automatically handles data alignment
shap_explanation = explainer(X_test_sample)

# Check shape to handle binary classification (Samples, Features, Classes)
# We want the values for Class 1 (the positive class)
if len(shap_explanation.shape) == 3:
    shap_values = shap_explanation[:, :, 1]
else:
    shap_values = shap_explanation

print("\nSHAP Summary Plot:")
# Pass the Explanation object directly. It contains both values and data.
shap.summary_plot(shap_values, X_test_sample)

print("\nSHAP Waterfall Plot (for a single prediction):")
# Pass the Explanation object for the first sample (index 0)
shap.waterfall_plot(shap_values[0])

```

```

[notice] A new release of pip is available: 24.0 -> 25.3
[notice] To update, run: C:\Users\Bhautik\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.
11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

```


Requirement already satisfied: shap in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (0.50.0)

Requirement already satisfied: numpy>=2 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from shap) (2.3.5)

Requirement already satisfied: scipy in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from shap) (1.17.0)

Requirement already satisfied: scikit-learn in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from shap) (1.8.0)

Requirement already satisfied: pandas in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from shap) (2.3.3)

Requirement already satisfied: tqdm>=4.27.0 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from shap) (4.67.1)

Requirement already satisfied: packaging>20.9 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from shap) (25.0)

Requirement already satisfied: slicer==0.0.8 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from shap) (0.0.8)

Requirement already satisfied: numba>=0.54 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from shap) (0.63.1)

Requirement already satisfied: cloudpickle in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from shap) (3.1.2)

Requirement already satisfied: typing-extensions in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from shap) (4.15.0)

Requirement already satisfied: llvmlite<0.47,>=0.46.0dev0 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from numba>=0.54->shap) (0.46.0)

Requirement already satisfied: colorama in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from tqdm>=4.27.0->shap) (0.4.6)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from pandas->shap) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from pandas->shap) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from pandas->shap) (2025.2)

Requirement already satisfied: joblib>=1.3.0 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from scikit-learn->shap) (1.5.3)

Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from scikit-learn->shap) (3.6.0)

Requirement already satisfied: six>=1.5 in c:\users\bhautik\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)

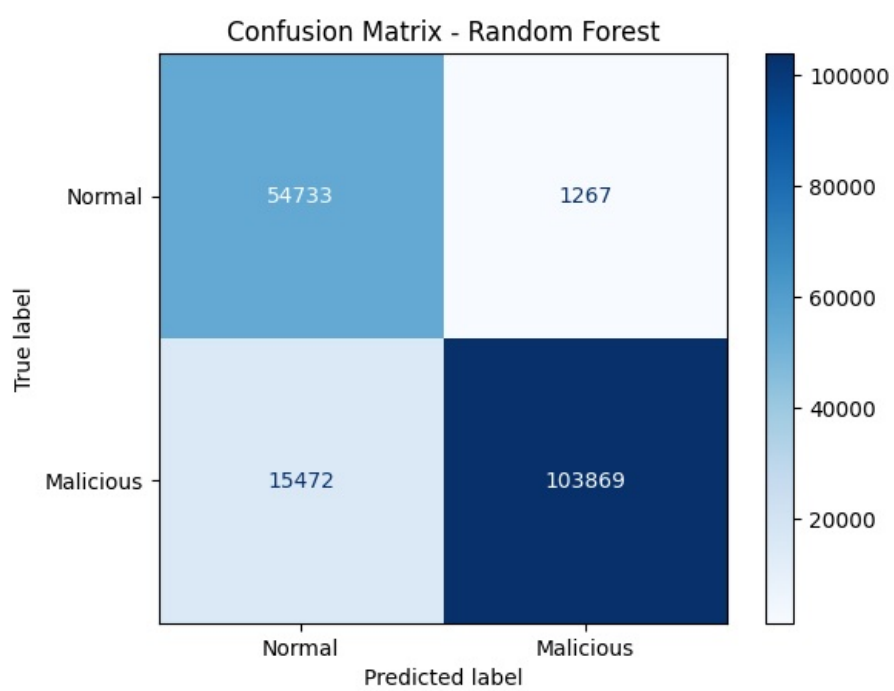
Classification Report:

	precision	recall	f1-score	support
0	0.78	0.98	0.87	56000
1	0.99	0.87	0.93	119341
accuracy			0.90	175341
macro avg	0.88	0.92	0.90	175341
weighted avg	0.92	0.90	0.91	175341

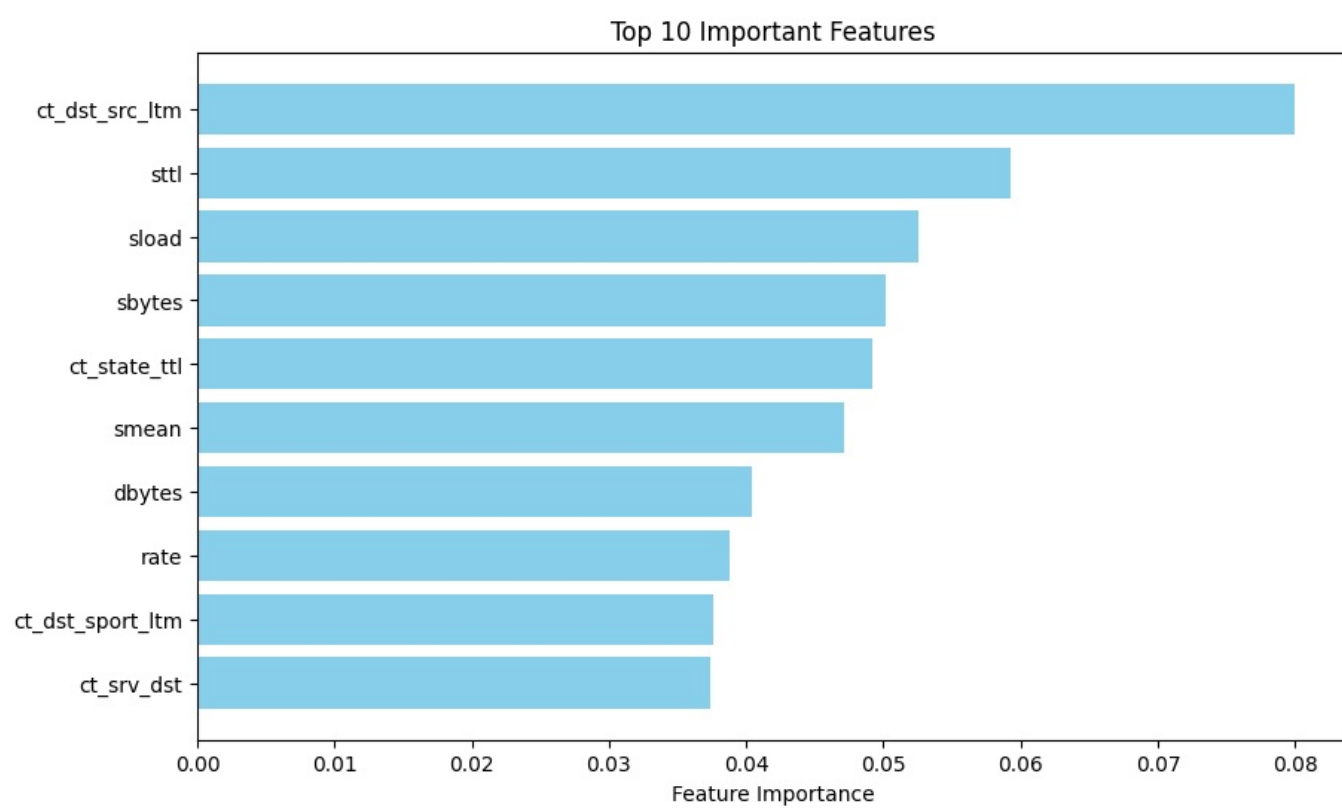
Confusion Matrix:

```
[[ 54733 1267]
 [ 15472 103869]]
```

<Figure size 800x600 with 0 Axes>



Accuracy: 0.9045



SHAP Summary Plot:

